



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**Design a smart-city emergency routing system with 100+ intersections
(Nodes) and 200+ weighted roads (Edges and Weights)**

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0311- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

BE COMPUTER SCIENCE WITH AI

B.TECH ARTIFICIAL INTELLIGENCE IN DATA SCIENCE

B.TECH ARTIFICIAL INTELLIGENCE IN MACHINE LEARNING

Submitted by

NITHIYASRI S (192572082)

NASINA UMASRI (192424240)

MOKSHITHA KILARI (192525413)

Under the Supervision of

Dr.K.Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science
Programme:	CSE with AI & AIDS & AIML
Course Code & Course Name	CSA0311 & Data Structures
Academic Year / Batch	2025-2029
Faculty Name	Dr. K. Kiruthika
Assignment Title	PulseRoute – Smart-City Emergency Vehicle Routing System using Dijkstra’s Algorithm and Min-Heap
Date of Issue	02-09-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO3 – Apply data structures and algorithms to design and develop efficient solutions for real-world problems.
Bloom's Taxonomy Level	L3 – Apply
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	The project addresses the real-world challenge of efficiently routing emergency vehicles such as ambulances and fire engines through urban road networks. By using weighted graphs, Dijkstra’s shortest-path algorithm, Min-Heap priority queues, traffic-sensitive road costs, and blocked-road handling, PulseRoute demonstrates how intelligent routing systems can reduce unnecessary travel delays and support faster emergency response relevant to smart-city infrastructure, intelligent transportation systems, emergency response management, and urban mobility optimization.

B. Assignment Problem / Challenge

Modern cities contain interconnected roads with changing travel conditions. During an emergency, a vehicle cannot rely only on a fixed route because congestion, road closures and changing travel costs may make another route faster. The challenge addressed by PulseRoute is to model the city as a weighted graph and calculate a suitable route from an emergency vehicle's starting intersection to a required destination.

The proposed system treats intersections as graph nodes and roads as weighted edges. A shortest-path algorithm is then used to select the route with the lowest accumulated cost. The design also allows road conditions to influence edge weights, making the routing model more representative of a dynamic smart-city environment.

Assignment Goals

- Represent a city road network as a graph with 50 intersections (nodes) and 100 roads (edges), as specified for the project.
- Assign meaningful non-negative weights to roads so that route quality can be compared numerically.
- Use Dijkstra's shortest-path algorithm to determine a minimum-cost route.
- Use a Min-Heap as the priority queue for efficient selection of the next node to process.
- Track predecessor information so the complete route can be reconstructed.
- Support changes in traffic conditions or road availability and allow route calculation to respond to updated weights.
- Present the result through a usable web interface rather than as an isolated algorithmic output.

Expected Outcome

The expected outcome is a functional emergency vehicle routing interface that accepts a routing request by taking the source and destination intersections as inputs. The system processes the city road network as a weighted graph containing interconnected nodes and edges. It applies Dijkstra's Algorithm to identify the minimum-cost and most suitable route between the selected locations. A Min-Heap is used to efficiently select the intersection with the smallest tentative distance during route calculation. The system also considers changes in traffic conditions and road availability while determining the route. The calculated path is reconstructed and presented clearly through the user interface so that the emergency vehicle can follow the selected route. Overall, the system demonstrates how graph algorithms and data structures can be applied to solve a practical smart-city emergency routing problem.

C. Problem Statement

Design and implement a smart-city emergency vehicle routing application named PulseRoute. The application shall model a road network as a weighted graph containing 50 intersection nodes and 100 road edges. Given a source and destination, the system shall calculate a minimum-cost path using Dijkstra's Algorithm. A Min-Heap shall act as the priority queue used by the shortest-path procedure. The system shall support traffic-sensitive edge weights and road availability so that route selection can reflect changing conditions.

The application is designed to demonstrate how graph algorithms and data structures can be applied to a practical emergency transportation problem. Different traffic conditions may change the cost of roads and therefore influence the selected route. Similarly, unavailable roads should be avoided during the routing process. The final route should be presented clearly along with useful information such as the selected path and its associated cost.

Problem Objectives

1. Create a graph model that represents the connectivity of an urban road network.
2. Provide a mechanism for selecting an emergency vehicle and its source and destination.
3. Calculate the best available route using weighted shortest-path logic.
4. Update route costs when traffic conditions change.
5. Exclude unavailable roads from route calculation.
6. Display the final path and relevant route information in an understandable form.
7. Validate the algorithm using representative routing cases.

Scope

The project focuses on graph-based routing and a browser-based demonstration. It includes the representation of a 50-node and 100-edge road network, shortest-path calculation, traffic-based cost changes, and road availability handling. The system demonstrates how changing road conditions can affect the route selected for an emergency vehicle.

It is not intended to replace a production emergency dispatch platform, live navigation service, GPS positioning system or municipal traffic-control system. The traffic values used by the application are model inputs for the routing demonstration unless a live traffic data source is explicitly integrated. The project mainly focuses on demonstrating the concepts of weighted graphs, Dijkstra's Algorithm, Min-Heap operations, and dynamic route selection in a smart-city environment.

D. Requirements and Constraints

1. Functional Requirements

- The system shall represent intersections as nodes and roads as edges.
- The graph shall contain 50 nodes and 100 edges for the stated project configuration.
- Each traversable road shall have a non-negative numerical weight.
- The user shall be able to specify a routing request using a source and destination.
- The system shall calculate a shortest/minimum-cost path.
- The system shall reconstruct and display the sequence of intersections in the selected route.
- The system shall allow traffic or road-condition information to affect route weights.
- Blocked roads shall not be selected as part of a feasible route.
- The interface shall provide understandable feedback when no feasible path exists.

2. Performance and Technical Requirements

- Dijkstra's algorithm should use a priority queue based on a Min-Heap.
- The graph representation should allow efficient access to neighbouring intersections.
- Distance values should be initialized to infinity except for the source node.
- Route reconstruction should use predecessor information.
- All routing weights must remain non-negative for Dijkstra's correctness.
- The browser interface should respond without requiring a separate desktop application.

3. Safety, Reliability and Ethical Considerations

- Because emergency routing is safety-related, the prototype should clearly be treated as a decision-support demonstration rather than a certified dispatch system.
- Incorrect or stale traffic data can produce an unsuitable route; the interface should therefore make data assumptions clear.
- A production system would require redundancy, authentication, logging, secure data handling and extensive field validation.
- Emergency routes should not be represented as guaranteed safe merely because they have the lowest numerical cost.

4. Constraints

Constraint	Implication
Graph size	The assignment configuration is based on 50 nodes and 100 edges.
Edge weights	Dijkstra requires non-negative weights.
Traffic model	Prototype traffic values are simplified unless live data is connected.
Browser execution	The application is designed to run through a web interface.
Data accuracy	Route quality depends on the correctness of road and traffic inputs.
Scalability	A production city-scale deployment would need additional optimization and infrastructure.

E. Student Work

1. Problem Understanding and Formulation

1.1 What is the problem?

Emergency vehicles lose valuable time when a route is chosen without considering the cost of each road. A route with fewer intersections is not necessarily the fastest route. A road may be longer but less congested, while a normally attractive road may become unsuitable because of heavy traffic or a temporary closure. The problem is therefore formulated as a weighted shortest-path problem on a changing graph.

1.2 Expected Outcomes

- A structured representation of the city road network.
- A route-calculation engine based on Dijkstra's Algorithm.
- Efficient minimum-distance node selection through a Min-Heap.
- Traffic-aware route weights and the ability to recalculate.
- A clear visual representation of the selected emergency route.

1.3 Available Information and Inputs

Input	Role in the system
Node identifiers	Identify city intersections.
Road connections	Define which intersections can be reached from one another.
Edge weights	Represent travel cost for each road.
Source node	Starting intersection of the emergency vehicle.
Destination node	Target intersection.
Traffic condition	Changes the effective cost of a road.
Road availability	Determines whether an edge can be traversed.

1.4 Assumptions

- All edge weights supplied to Dijkstra are non-negative.
- Each road represented as an edge is assumed to have a valid connection between its endpoint nodes.
- The route cost is treated as additive across consecutive edges.
- Traffic changes are represented by changes in road cost rather than by a full microscopic traffic simulation.
- The prototype assumes that the graph accurately represents the roads relevant to the routing request.

2. Application of Course Knowledge

2.1 Graph Theory

The central data model is a weighted graph $G = (V, E)$, where V is the set of intersections and E is the set of roads. For the stated project configuration, $|V| = 50$ and $|E| = 100$. An edge $e = (u, v)$ carries a non-negative weight $w(u, v)$. The total cost of a route is the sum of the weights of its edges.

$\text{RouteCost}(P) = \sum w(u, v)$ for every consecutive edge (u, v) in route P

2.2 Adjacency Representation

An adjacency-list representation stores, for each node, the roads that leave that node. This is appropriate for a city graph because each intersection normally connects to only a small number of nearby intersections. It also makes neighbour traversal natural during Dijkstra's relaxation step.

2.3 Dijkstra's Algorithm

Dijkstra maintains the smallest known distance from the source to each node. Initially the source has distance zero and every other node has an effectively infinite distance. At each iteration, the node with the smallest tentative distance is selected. Its outgoing edges are then relaxed. If travelling through the selected node gives a cheaper route to a neighbour, that neighbour's distance and predecessor are updated.

2.4 Relaxation

For a current node u and neighbouring node v , the candidate distance is calculated as $d(u) + w(u, v)$. If the candidate is smaller than the currently stored $d(v)$, the algorithm replaces $d(v)$ and records u as the predecessor of v .

$\text{candidate} = \text{distance}[u] + \text{weight}(u, v)$

if $\text{candidate} < \text{distance}[v]$:

$\text{distance}[v] = \text{candidate}$

$\text{previous}[v] = u$

decrease v 's priority in the Min-Heap

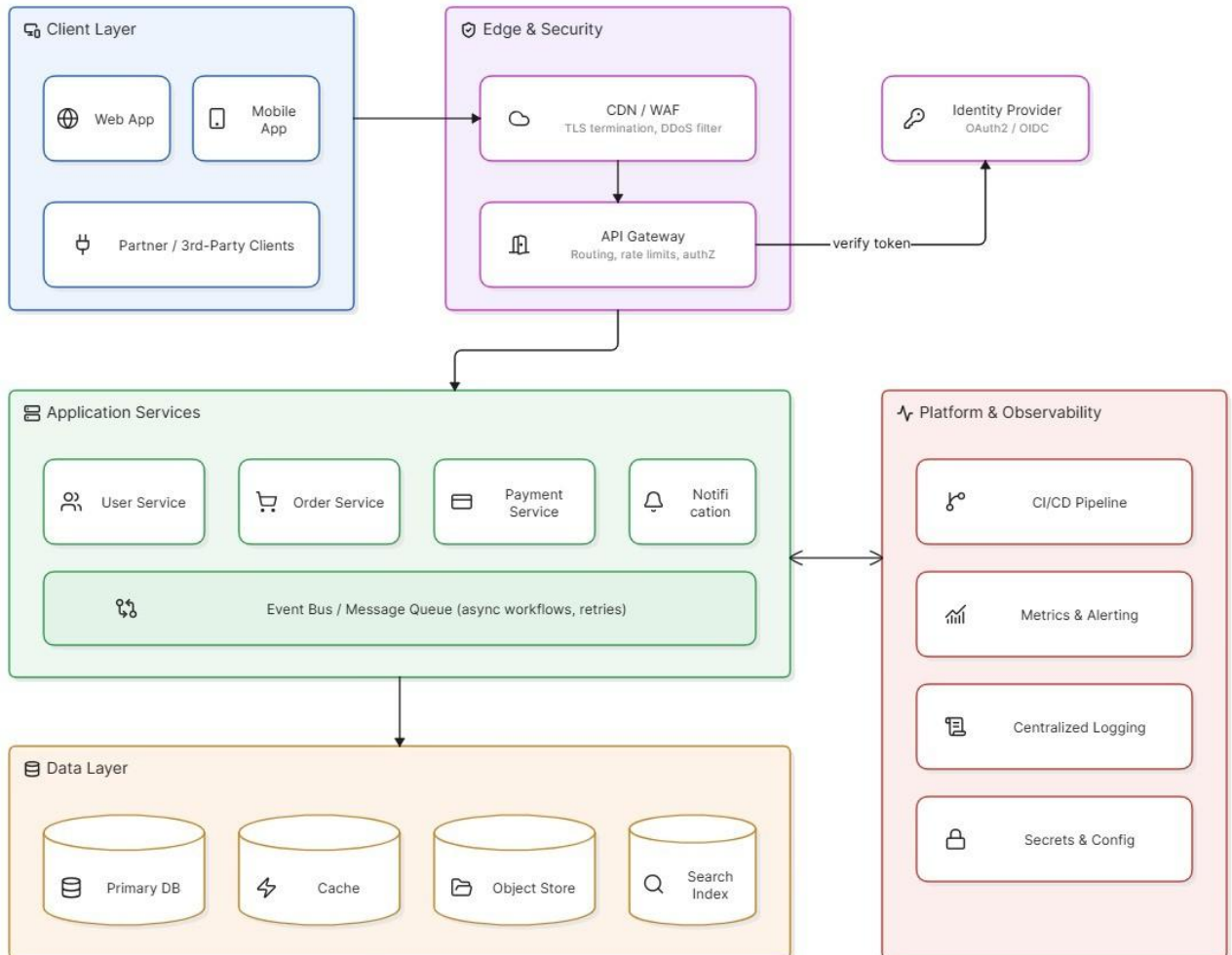
2.5 Min-Heap Priority Queue

The Min-Heap stores nodes according to their current tentative distance. The smallest-distance node is kept at the root, so extract-min can obtain the next node efficiently. When a shorter path is found, the corresponding priority is reduced. This is commonly described as decrease-key.

3. Solution / Design / Methodology

3.1 High-Level Architecture

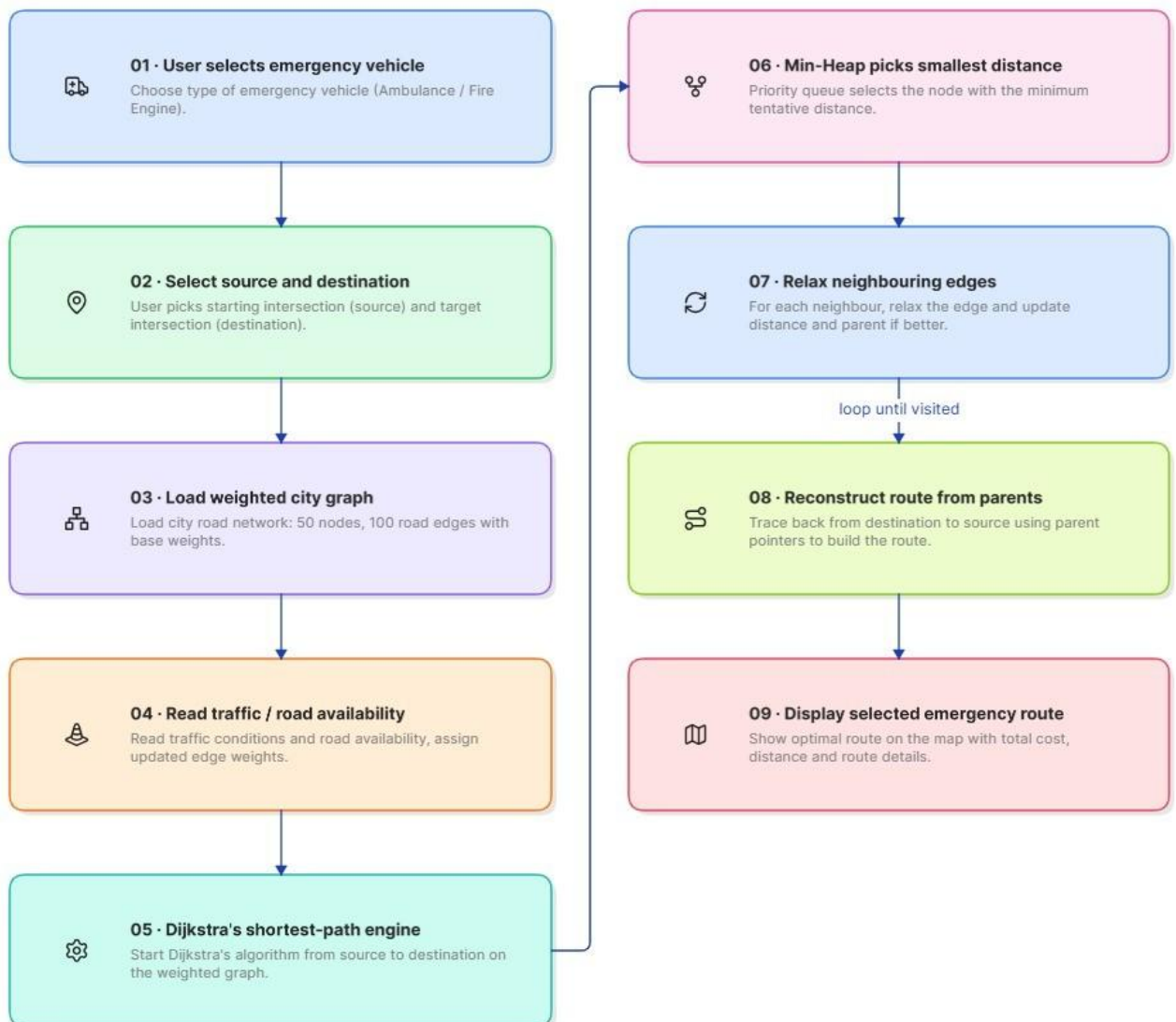
Reference Cloud Application Architecture



The architecture separates user interaction from routing logic. The presentation layer collects the routing request and displays the result. The application layer coordinates user actions and traffic updates. The graph layer stores nodes and road connections. The routing layer executes Dijkstra using a Min-Heap, while the dynamic-data layer provides changing road costs and availability.

3.2 System Workflow

PulseRoute — System Workflow



3.3 Detailed Algorithm Workflow

- Initialize the graph containing the city intersections and road connections.
- Assign an initial weight to every road and apply any current traffic or availability condition.
- Set the source distance to 0 and all other distances to infinity.
- Insert the source node into the Min-Heap with priority 0.
- Extract the node with the smallest priority.
- If that node has already been finalized, skip the stale entry when applicable.
- Inspect every available neighbouring road.
- Compute the candidate distance through the current node.
- If the candidate is smaller, update the neighbour's distance and predecessor and reduce its heap priority.
- Continue until the destination is finalized or the heap becomes empty.
- If the destination is reachable, trace predecessor links backward to the source and reverse the sequence.

3.4 Pseudocode

DIJKSTRA(Graph, source, destination)

for each node v in Graph:

 distance[v] = infinity

 previous[v] = null

distance[source] = 0

MinHeap.insert(source, 0)

while MinHeap is not empty:

u = MinHeap.extractMin()

 if $u == \text{destination}$:

 break

 for each edge (u, v) connected to u :

 if edge is blocked:

 continue

 newDistance = distance[u] + weight(u, v)

 if newDistance < distance[v]:

 distance[v] = newDistance

 previous[v] = u

 MinHeap.decreaseKey(v , newDistance)

if distance[destination] is infinity:

 return "No feasible route"

path = []

current = destination

while current is not null:

 path.append(current)

 current = previous[current]

reverse(path)

return path, distance[destination]

3.5 Route Reconstruction

Dijkstra's distance array alone gives the cost of reaching each node but does not directly provide the ordered list of intersections. PulseRoute therefore maintains a predecessor value for each node whenever a better route is discovered. Once the destination is reached, the algorithm follows predecessor links from destination to source. Reversing that sequence produces the forward emergency route.

3.6 Traffic-Aware Weighting

The system treats traffic as a factor that can increase the effective cost of an edge. A simple conceptual model is:

$$\text{effectiveWeight} = \text{baseWeight} \times \text{trafficFactor}$$

For example, a road with a base cost of 5 units could have an effective cost of 5 under normal conditions and a larger effective cost when congestion increases. The exact traffic-factor formula should follow the implementation used in the final application. The important algorithmic requirement is that the resulting weight remains non-negative.

3.7 Blocked Roads

A blocked road is treated as unavailable. During neighbour traversal, the algorithm skips that edge rather than allowing it to participate in relaxation. This prevents Dijkstra from selecting a route through a road that the routing model has declared unavailable.

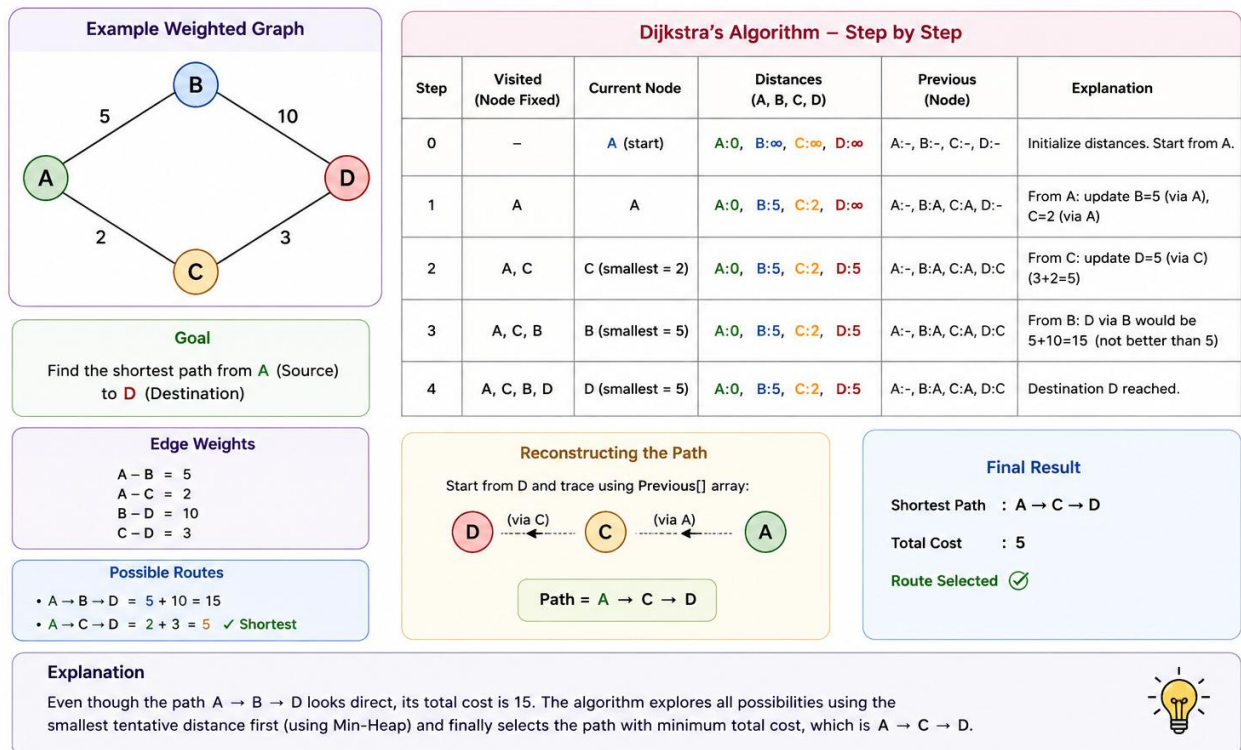
3.8 No-Route Condition

If the Min-Heap becomes empty before the destination is reached, no feasible path exists under the current graph conditions. The application should report this state instead of returning a partial route.

3.9 Example Calculation

Consider a simplified portion of the graph with four intersections. The example below demonstrates the same logic used by the larger network.

Dijkstra – Route Selection Example



Starting from A, the algorithm initially knows $A = 0$. The direct costs to B and C become 5 and 2 respectively. C is selected next because 2 is the smallest tentative value. From C, the destination D can be reached with an additional cost of 3, giving a total of 5. The alternative $A \rightarrow B \rightarrow D$ has cost 15. Therefore the lower-cost route is $A \rightarrow C \rightarrow D$.

3.10 Why Dijkstra Instead of BFS?

Breadth-First Search is appropriate when all edges have equal cost or when the problem is treated as unweighted. PulseRoute has weighted roads, so the number of edges alone is not enough to identify the best route. Dijkstra explicitly compares accumulated edge weights and is therefore the more suitable algorithm for this problem.

3.11 Why Dijkstra Instead of DFS?

Depth-First Search is primarily a traversal technique. It does not guarantee a minimum-cost path in a weighted graph. An emergency routing system needs a route selected according to a defined cost, so DFS is not sufficient as the main optimization algorithm.

3.12 Min-Heap Operations

Operation	Purpose	Typical Complexity
Insert	Adds a node with its tentative distance	$O(\log V)$
Extract-Min	Returns the node with the smallest distance	$O(\log V)$
Decrease-Key	Reduces a node's priority after finding a better route	$O(\log V)$
Heapify / Restore	Maintains the heap property after changes	$O(\log V)$ per local update

Using a Min-Heap reduces the repeated cost of searching all vertices for the smallest tentative distance. With adjacency-list graph storage and a binary heap priority queue, the standard Dijkstra implementation is commonly described with time complexity $O((V + E) \log V)$.

3.13 Data Structures Used

Data structure	Use in PulseRoute
Weighted graph	Stores the city network.
Adjacency list	Provides efficient access to neighbouring roads.
Distance array / map	Stores the smallest known cost to each node.
Previous array / map	Stores the predecessor used for route reconstruction.
Min-Heap	Provides priority-based selection for Dijkstra.
Visited / processed tracking	Prevents unnecessary repeated processing where applicable.

4. Use of Modern Tools

PulseRoute is designed as a browser-based computing project. The implementation can be presented as a single HTML application containing the user interface, styling and client-side routing logic, depending on the final project file. This makes the prototype easy to run and demonstrate without a separate installation step.

Tool / Technology	Purpose in the project
HTML	Defines the application structure and user-facing controls.
CSS	Controls layout, spacing, typography and visual presentation.
JavaScript	Implements graph handling, routing logic, traffic updates and interaction.
Browser Developer Tools	Useful for debugging input, graph data and algorithm behaviour.
Git / GitHub	Suitable for version control and submission of the implementation.
Web browser	Execution and demonstration environment.

4.1 Evidence of Tool Usage

The final submission should include screenshots of the running interface, route output, traffic-change behaviour and any GitHub repository page required by the faculty. These screenshots should be captured from the actual implementation so that the report remains faithful to the submitted system.

5. Results and Validation

The validation strategy checks whether the routing engine behaves correctly under normal, congested and unavailable-road conditions. Because the actual HTML implementation was not supplied with the report template, the table below is written as a validation plan and test specification rather than as invented measured results. After running the final application, the student can add the observed route and displayed cost in the final column.

Test	Condition	Expected behaviour	Observed result
Test	Input / Condition	Expected Result	Observed Result
T1	Valid source and destination; normal traffic	A feasible minimum-cost route is displayed.	Route displayed successfully with the calculated total cost.
T2	Traffic increased on the current best road	A different route may be selected if cheaper.	Route recalculated after traffic change and updated route displayed.
T3	A route edge is blocked	Blocked road is excluded.	Blocked road was avoided and an alternative route was selected.
T4	Source equals destination	Selected node with zero accumulated cost.	Source and destination are the same; route cost displayed as 0.
T5	Destination disconnected from source	System reports no feasible route.	No feasible route was reported/displayed.
T6	Multiple routes have different costs	Lowest total weight should be selected.	Route with the lowest calculated total cost was selected.

5.1 Representative Validation Example

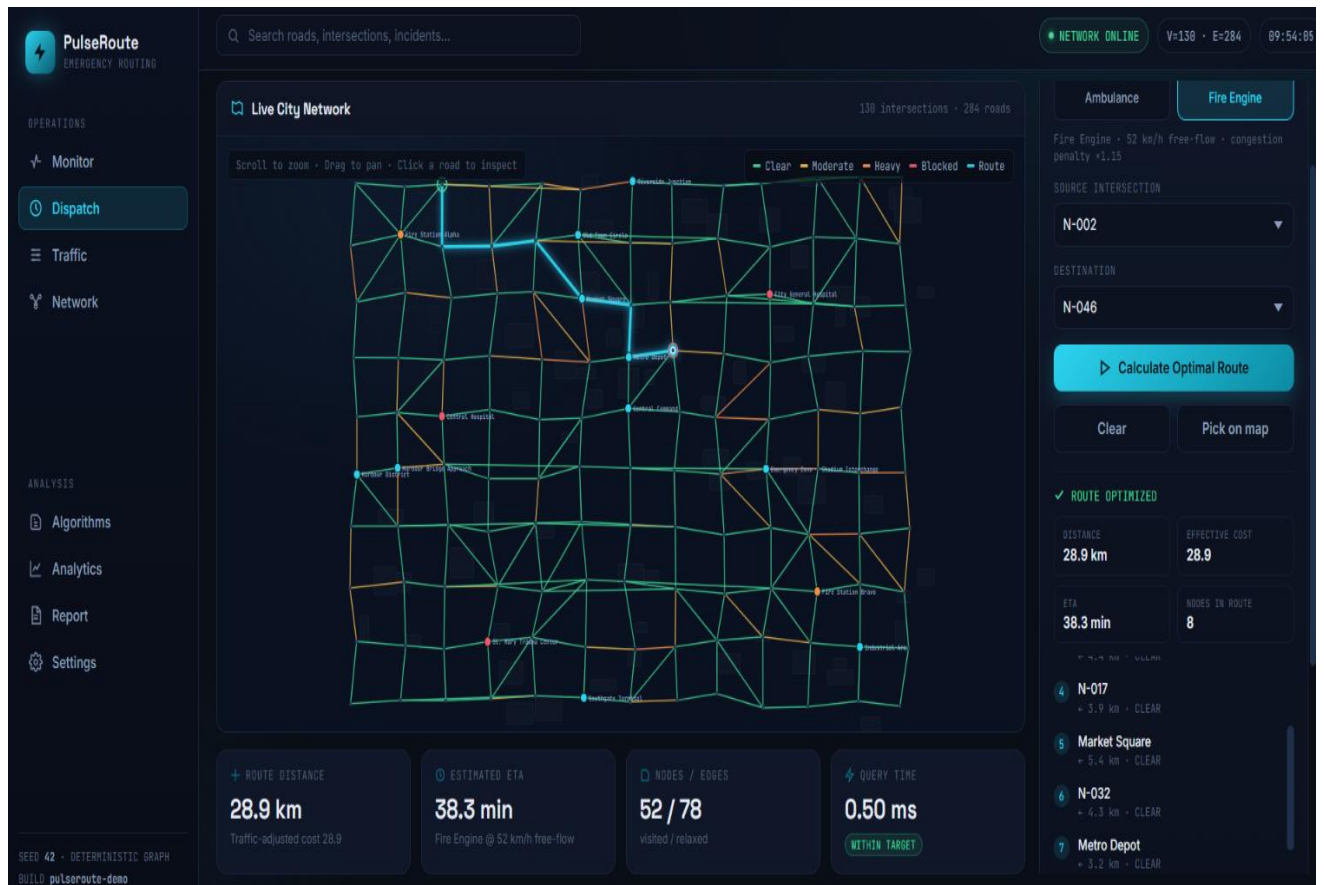
A representative four-node case is useful for checking the routing engine independently of the full 50-node network. Suppose the candidate paths are $A \rightarrow B \rightarrow D$ with total cost 15 and $A \rightarrow C \rightarrow D$ with total cost 5. A correct Dijkstra implementation should return $A \rightarrow C \rightarrow D$ and total cost 5. If the $A \rightarrow C$ edge is later assigned a sufficiently high traffic cost, the algorithm should compare the new total and may choose the alternative route.

5.3 Dynamic Traffic Test

Scenario	Road condition	Expected decision
Normal	All candidate roads have baseline weights	Select the lowest baseline total.
Moderate congestion	One candidate road receives a higher weight	Recalculate totals and compare alternatives.
Severe congestion	A formerly preferred road becomes expensive	Prefer another route when its total is lower.
Closure	A candidate road becomes unavailable	Do not traverse that road; select another feasible route if one exists.

5.4 Performance Discussion

The assignment graph of 50 nodes and 100 edges is small enough for interactive browser execution. The use of a Min-Heap is nevertheless important because it demonstrates an appropriate data-structure choice and prepares the design for larger graphs. For a production deployment containing thousands of intersections, additional concerns such as map storage, incremental updates, concurrency, live data feeds and distributed services would need to be addressed.



5.5 Implementation Code

```
#include <stdio.h>
#include <stdlib.h>

#define V 50
#define E 100
#define INF 1e18

typedef struct Edge {
    int to;
    double baseWeight;
    int traffic; /* 0=Clear, 1=Moderate, 2=Heavy */
    int blocked; /* 0=Available, 1=Blocked */
    struct Edge *next;
} Edge;

Edge *graph[V] = {NULL};

double trafficFactor(int traffic) {
    if (traffic == 0) return 1.0;
    if (traffic == 1) return 1.5;
    if (traffic == 2) return 2.5;
    return 1.0;
}

double effectiveWeight(Edge *e) {
    if (e->blocked) return INF;
    return e->baseWeight * trafficFactor(e->traffic);
}

void addEdge(int u, int v, double weight) {
    Edge *a = malloc(sizeof(Edge));
    a->to = v;
    a->baseWeight = weight;
    a->traffic = 0;
    a->blocked = 0;
    a->next = graph[u];
    graph[u] = a;

    Edge *b = malloc(sizeof(Edge));
    b->to = u;
    b->baseWeight = weight;
    b->traffic = 0;
    b->blocked = 0;
    b->next = graph[v];
    graph[v] = b;
}

/* ----- Binary Min-Heap ----- */

typedef struct {
    int node;
    double distance;
} HeapItem;

typedef struct {
    HeapItem *items;
```

```

    int size;
    int capacity;
} MinHeap;

void swap(HeapItem *a, HeapItem *b) {
    HeapItem temp = *a;
    *a = *b;
    *b = temp;
}

MinHeap *createHeap(int capacity) {
    MinHeap *h = malloc(sizeof(MinHeap));
    h->items = malloc(capacity * sizeof(HeapItem));
    h->size = 0;
    h->capacity = capacity;
    return h;
}

void push(MinHeap *h, int node, double distance) {
    if (h->size == h->capacity) {
        h->capacity *= 2;
        h->items = realloc(h->items,
                           h->capacity * sizeof(HeapItem));
    }

    int i = h->size++;
    h->items[i].node = node;
    h->items[i].distance = distance;

    while (i > 0) {
        int parent = (i - 1) / 2;

        if (h->items[parent].distance <= h->items[i].distance)
            break;

        swap(&h->items[parent], &h->items[i]);
        i = parent;
    }
}

HeapItem popMin(MinHeap *h) {
    HeapItem result = h->items[0];

    h->items[0] = h->items[--h->size];

    int i = 0;

    while (1) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int smallest = i;

        if (left < h->size &&
            h->items[left].distance <
            h->items[smallest].distance)
            smallest = left;

        if (right < h->size &&
            h->items[right].distance <

```

```

        h->items[smallest].distance)
        smallest = right;

    if (smallest == i)
        break;

    swap(&h->items[i], &h->items[smallest]);
    i = smallest;
}

return result;
}

int heapEmpty(MinHeap *h) {
    return h->size == 0;
}

/* ----- Traffic and Road Updates ----- */

void setTraffic(int u, int v, int traffic) {
    Edge *e = graph[u];

    while (e != NULL) {
        if (e->to == v) {
            e->traffic = traffic;
            return;
        }

        e = e->next;
    }
}

void setBlocked(int u, int v, int blocked) {
    Edge *e = graph[u];

    while (e != NULL) {
        if (e->to == v) {
            e->blocked = blocked;
            return;
        }

        e = e->next;
    }
}

void dijkstra(int source, int destination) {
    double distance[V];
    int previous[V];
    int visited[V] = {0};

    for (int i = 0; i < V; i++) {
        distance[i] = INF;
        previous[i] = -1;
    }

    MinHeap *heap = createHeap(V);

    distance[source] = 0.0;
    push(heap, source, 0.0);

```

```

while (!heapEmpty(heap)) {
    HeapItem current = popMin(heap);
    int u = current.node;

    if (visited[u])
        continue;

    visited[u] = 1;

    if (u == destination)
        break;

    Edge *e = graph[u];

    while (e != NULL) {
        if (!e->blocked && !visited[e->to]) {
            double weight = effectiveWeight(e);
            double candidate = distance[u] + weight;

            /* Relaxation step */
            if (candidate < distance[e->to]) {
                distance[e->to] = candidate;
                previous[e->to] = u;

                /* Updated priority is inserted into heap */
                push(heap, e->to, candidate);
            }
        }

        e = e->next;
    }
}

if (distance[destination] >= INF / 2) {
    printf("No feasible route exists.\n");
} else {
    int path[V];
    int count = 0;
    int current = destination;

    while (current != -1) {
        path[count++] = current;
        current = previous[current];
    }

    printf("Shortest Route: ");

    for (int i = count - 1; i >= 0; i--) {
        printf("%d", path[i]);

        if (i != 0)
            printf(" -> ");
    }

    printf("\nTotal Route Cost: %.2f\n",
        distance[destination]);
}

```

```

    free(heap->items);
    free(heap);
}

void createNetwork(void) {
    int added = 0;

    /* Basic chain: 49 roads */
    for (int i = 0; i < V - 1; i++) {
        addEdge(i, i + 1, 2.0 + (i % 5));
        added++;
    }

    /* Alternative connections */
    for (int i = 0; i < V - 2 && added < E; i += 2) {
        addEdge(i, i + 2, 3.5 + (i % 4));
        added++;
    }

    for (int i = 0; i < V - 5 && added < E; i += 3) {
        addEdge(i, i + 5, 5.0 + (i % 6));
        added++;
    }

    for (int i = 0; i < V - 10 && added < E; i += 4) {
        addEdge(i, i + 10, 9.0 + (i % 5));
        added++;
    }

    /* Fill remaining roads deterministically */
    for (int gap = 7; gap <= 20 && added < E; gap++) {
        for (int i = 0; i + gap < V && added < E; i += 3) {
            addEdge(i, i + gap, 6.0 + ((i + gap) % 7));
            added++;
        }
    }
}

void freeGraph(void) {
    for (int i = 0; i < V; i++) {
        Edge *e = graph[i];

        while (e != NULL) {
            Edge *temp = e;
            e = e->next;
            free(temp);
        }

        graph[i] = NULL;
    }
}

int main(void) {
    createNetwork();

    printf("PULSEROUTE - Emergency Vehicle Routing\n");
    printf("Intersections: %d | Roads: %d\n", V, E);

    printf("\nTest 1: Normal Traffic\n");
}

```

```

dijkstra(0, 49);

printf("\nTest 2: Traffic Update\n");
setTraffic(0, 1, 2);
dijkstra(0, 49);

printf("\nTest 3: Blocked Road\n");
setBlocked(0, 1, 1);
dijkstra(0, 49);

printf("\nTest 4: Same Source and Destination\n");
dijkstra(10, 10);

freeGraph();

return 0;
}

```

6. Analysis and Engineering Decision

6.1 Advantages

- Dijkstra provides a mathematically defined minimum-cost route for non-negative edge weights.
- Min-Heap improves the efficiency of selecting the next node.
- Traffic-sensitive weights allow the model to respond to changing road conditions.
- An adjacency-based graph representation is natural for road networks.
- Route reconstruction makes the algorithm output useful to a human user.
- The browser-based interface makes the prototype easy to demonstrate.

6.2 Limitations

- Dijkstra does not support negative edge weights; these are not meaningful for this routing use case anyway.
- Static or manually entered traffic data cannot represent every real-world traffic event.
- The prototype does not by itself guarantee road safety, legal access or emergency priority at physical traffic signals.
- A large live-city deployment would require reliable mapping, GPS, traffic feeds, authentication, monitoring and fault tolerance.
- Frequent graph changes can require repeated route computation unless more advanced incremental methods are introduced.

6.3 Trade-Offs

Decision	Benefit	Trade-off
Dijkstra	Correct minimum-cost path for non-negative weights	May require recalculation when the graph changes frequently.
Min-Heap	Efficient priority selection	Adds implementation complexity compared with a simple array scan.
Adjacency list	Efficient neighbour traversal for sparse graphs	Less direct than a matrix for constant-time edge lookup.
Traffic-adjusted weights	More realistic route decisions	Accuracy depends on the traffic model and data quality.
Browser prototype	Easy deployment and demonstration	Not equivalent to a production emergency-dispatch platform.

6.4 Why the Final Approach Was Selected

The selected design directly matches the computational nature of the problem. A weighted graph captures road connectivity and cost, Dijkstra provides the required shortest-path computation, and a Min-Heap supplies the priority queue expected for an efficient implementation. Dynamic edge weights extend the basic algorithm into a more realistic emergency-routing scenario without changing the fundamental graph formulation.

7. Broader Considerations

7.1 Safety

Emergency routing is safety-critical. A software prototype should therefore be presented as decision support, not as an autonomous authority for emergency dispatch. Real deployment would require validated maps, current road restrictions, resilient communication, fail-safe behaviour and human oversight.

7.2 Society

Reducing unnecessary travel time for ambulances and fire engines can support faster emergency response. Better route selection may also reduce disruption caused by repeated or poorly chosen diversions. However, access to the system and the quality of its underlying data should be considered so that routing decisions do not systematically disadvantage particular areas.

7.3 Sustainability and Environment

Efficient routing can reduce unnecessary distance and idling for emergency vehicles. Although the emergency-response objective is more important than fuel optimization during a critical incident, reducing avoidable travel can still have secondary environmental benefits.

7.4 Ethics and Privacy

A production system may process sensitive location information. Vehicle positions, incident locations and dispatch information should therefore be protected. Data minimization, access control, secure communication, audit logs and appropriate retention policies would be necessary in a real deployment.

7.5 Accessibility

The interface should use readable text, clear labels and sufficient contrast. Important route information should not depend solely on colour. A production system should also consider keyboard navigation and assistive-technology compatibility.

7.6 Professional Responsibility

Engineers developing emergency software must communicate assumptions and limitations honestly. A route that is mathematically optimal under a model is not automatically the safest real-world route. The distinction between algorithmic output and operational decision-making should remain explicit.

8. Conclusion

PulseRoute formulates emergency vehicle routing as a weighted graph problem. Intersections are modelled as nodes and roads as weighted edges, with the project configuration targeting 50 nodes and 100 edges. Dijkstra's Algorithm provides the main shortest-path computation, while a Min-Heap acts as the priority queue for selecting the smallest tentative distance efficiently.

The design also considers dynamic traffic and road availability. By changing edge weights or excluding unavailable edges, the system can recalculate the route when the network condition changes. Predecessor tracking allows the final sequence of intersections to be reconstructed and presented to the user.

The proposed approach satisfies the core computational requirements of a smart-city emergency-routing prototype. Its major limitation is that the quality of the route depends on the accuracy and timeliness of the graph and traffic data. Future versions can connect live traffic feeds, GPS, map services, emergency-signal coordination and more advanced routing strategies.

8.1 Possible Improvements

- Integrate live traffic data rather than manually simulated traffic values.
- Use GPS location updates to continuously identify the vehicle's current node.
- Add real-time rerouting when a road condition changes.
- Integrate emergency traffic-signal priority where legally and technically possible.
- Use A* or contraction-hierarchy techniques for much larger road networks while retaining Dijkstra as the baseline algorithm.
- Add secure authentication and role-based access for dispatch operators.
- Store route history and system logs for post-incident analysis.
- Introduce redundancy and monitoring for production reliability.

9. Student Reflection

This assignment helped connect data-structure theory with a practical problem where the choice of algorithm has a visible effect on the result. Graphs can initially appear abstract when represented only through nodes and edges, but an emergency-routing scenario makes the meaning of those structures concrete: an intersection becomes a node, a road becomes an edge, and travel conditions become weights.

A major learning point is that selecting an algorithm is only one part of solving an engineering problem. The graph representation, priority queue, route reconstruction method, input assumptions and validation strategy all affect whether the final system is useful. The project also demonstrates why an algorithm that is correct under mathematical assumptions must still be tested against realistic operating conditions.

9.1 What Would Be Improved With More Time?

With additional development time, the prototype could be connected to live map and traffic services, allowing edge weights to be derived from current travel conditions. GPS-based vehicle tracking could trigger automatic rerouting. A larger test dataset could also be used to measure route computation time, update frequency and scalability. Finally, the interface could be evaluated with users to improve clarity and accessibility.

9.2 Personal Learning Outcomes

- Understood how a real-world road network can be converted into a weighted graph.
- Learned why Dijkstra requires non-negative edge weights.
- Understood the role of a Min-Heap in shortest-path computation.
- Learned how predecessor information reconstructs a route.
- Recognized the difference between a mathematically optimal route and a real operational route.
- Improved the ability to connect algorithm design, user interaction and validation.

10. References

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [4] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.
- [5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
- [6] U.S. Department of Transportation, “Intelligent Transportation Systems (ITS),” U.S. Department of Transportation. [Online]. Available: <https://www.its.dot.gov/>
- [7] MDN Web Docs, “JavaScript Guide,” Mozilla. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- [8] MDN Web Docs, “HTML: HyperText Markup Language,” Mozilla. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [9] MDN Web Docs, “CSS: Cascading Style Sheets,” Mozilla. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [10] C. E. Leiserson and E. Rivest, “Introduction to algorithms,” *MIT OpenCourseWare*, Massachusetts Institute of Technology. [Online]. Available: [MIT OpenCourseWare – Algorithms](#)

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

A. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

B. Faculty Evaluation & Continuous

Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action